

DIPARTIMENTO DI INGEGNERIA INFORMATICA, AUTOMATICA
E GESTIONALE

Parameter-Efficient Few-Step Distillation of Diffusion Models via LoRA

COMPUTER VISION — PROJECT 5

Lightweight Diffusion Models for Resource-Constrained Environments

Professor:
Irene Amerini

Students:
Claudia Giuseppini — 2069387
Benedetta Conti — 2045621
Fabio Falconi — 2048079

Contents

1	Introduction	2
2	Related Work and Identified Limitations	2
3	Experimental Setup	2
4	Method	3
4.1	How diffusion models work	3
4.2	The forward noising process	3
4.3	The denoising network: an attention U-Net (standard)	4
4.4	The training objective	5
4.5	Engineering optimizations we added around the baseline	5
4.6	Fast sampling with DDIM	6
4.7	LoRA few-step distillation	6
5	Results	7
5.1	Baseline training	7
5.2	Quality and speed vs. number of steps	7
5.3	Novelty: matched-step comparison	8
5.4	Ablation 1 (causal): distillation objective vs. naive eps-loss	8
5.5	Ablation 2: LoRA rank ($K = 8$)	9
5.6	Ablation 3: distillation state source ($K = 8$)	9
6	Limitations	9
7	Conclusion	9

1 Introduction

Denoising Diffusion Probabilistic Models (DDPMs) [1] generate images by learning to gradually remove noise. They produce high-quality results, but generating a single image requires running the network many times in sequence (up to one thousand times), which makes them slow and expensive. This is a limitation when we want to run them on small devices or train them with limited hardware.

This project implements a standard DDPM baseline on CIFAR-10 and then integrates a distillation designed to make sampling faster. We evaluate the trade-off between computational efficiency (sampling time) and generation quality (FID, Inception Score), and we also measure sample diversity (Precision/Recall).

A faster deterministic sampler called DDIM [2] can reduce the number of steps, but quality collapses once the steps become very few. The reason is not that the network predicts noise badly at those steps: it is the *discretization error* that accumulates when we take large jumps along the denoising path. The distillation method we implemented mitigates exactly this error, while keeping the extra training cheap:

Parameter-efficient few-step distillation via LoRA. We freeze the trained DDPM parameters (the *teacher*) and train only small low-rank adapters [4] inserted into the attention layers (the *student*) with a different objective [3]. The student learns to reproduce, in very few steps, what the teacher produces with many DDIM sub-steps.

Because only the adapters are trained (about 0.1% of the weights) and they can be merged back into the network at inference, the method adds essentially no inference overhead while giving large speed-ups over the full-step baseline.

2 Related Work and Identified Limitations

- **DDPM** [1] — our baseline generative model. *Limitation:* very slow sampling ($T = 1000$ sequential network evaluations).
- **DDIM** [2] — a deterministic sampler that allows fewer steps. *Limitation:* quality collapses under aggressive step reduction (in our measurements FID rises from ~ 29 at 100 steps to >100 at 4 steps).
- **Progressive Distillation** [3] — distils a teacher into a student that halves the number of steps. *Limitation:* it re-trains the *whole* network at each stage, which is costly on limited hardware.
- **LoRA** [4] — low-rank adapters for cheap, parameter-efficient adaptation of a frozen model.

Our novelty sits at the intersection of these works: we bring progressive-distillation-style trajectory matching to diffusion, *but we perform it with LoRA only*. This addresses the cost limitation of distillation while attacking the discretization-error limitation of few-step DDIM.

3 Experimental Setup

This section lists the concrete configuration of our experiments.

Data. CIFAR-10: 60,000 colour images of size 32×32 with 3 channels (RGB), split into 50,000 for training and 10,000 for testing, across 10 classes. Pixel values are rescaled from $[0, 1]$ to $[-1, +1]$ so that the data is centred on zero, which matches the zero-centred Gaussian noise used by the diffusion process.

Model and training. An attention U-Net with `base_channels= 64` and `time_dim= 256` (15.3M parameters). We use `T=1000` diffusion steps with a linear noise schedule, the AdamW optimizer, batch size 1.

Distillation. For each target step budget $K \in \{8, 4, 2, 1\}$ we train one student for 10 epochs. Only the LoRA adapters are trained: 16,384 parameters out of 15.3M, i.e. about 0.107% of the model.

Metrics. We report FID and KID (image fidelity), Inception Score (IS, sharpness/recognisability), Precision and Recall [5] (fidelity vs. diversity), and seconds per image (speed). All metrics are computed on 10 000 generated images, compared against the CIFAR-10 test set with the `torch-fidelity` library, and every method starts from the *same* seeded initial noise so the comparisons are paired.

Hardware and reproducibility. Experiments run on a single GPU (NVIDIA T4 on Kaggle). We fix all random seeds (`seed = 42`), enable deterministic algorithms, seed the data-loader workers and the sampling noise generator, and export the resolved configuration and the exact package versions.

4 Method

We first explain the standard DDPM model following the standard from the literature, then the engineering optimizations we added around it, and finally the distillation.

4.1 How diffusion models work

A diffusion model is built on two opposite processes. The *forward* process takes a real image and adds a little random noise, step after step, until after $T = 1000$ steps the image is pure noise. This process is fixed and requires no learning. The *reverse* process is where a neural network learns to remove a little noise at a time. Once trained, we can start from pure noise and apply the network repeatedly to “uncover” a new image. In short, generating an image means denoising repeatedly.

The reason we use so many small steps is that each single step is easy: from a slightly noisy image to a slightly-less-noisy one, the change is tiny and the network can predict it accurately. Asking the network to jump from pure noise to a finished image in one shot would be almost impossible, because many different images could explain the same noise, and the average answer is blurry.

4.2 The forward noising process

The forward process is described by a single formula that directly gives the noisy image at any level t , without simulating all the intermediate steps. The closed formula is possible because the diffusion is modeled like a Markov process where every state depends entirely only on the previous state. Therefore we can predict the n -th state directly if we have the first one. The closed formula follows:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \quad (1)$$

The noisy image x_t is a weighted mix of two things: the clean image x_0 and pure random noise ε . The weight $\bar{\alpha}_t$ (“alpha-bar at t ”) is a number between roughly 1 and 0 that decides how much of each we keep. At small t , $\bar{\alpha}_t$ is close to 1, so the image dominates and there is little noise; at large t , it approaches 0, so noise dominates. The value $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ is just the running

product of the “keep” factors, where each β_s says how much noise is added at step s . We use a *linear schedule*, meaning the β_s grow linearly from a small start value to a larger end value.

4.3 The denoising network: an attention U-Net (standard)

The network that removes noise is a **U-Net** [1]. Its name comes from its shape that is like the letter U: on the way down it *compresses* the image (reduces the spatial size and increases the number of channels, capturing the overall context); on the way up it *expands* it back to the original size (rebuilding the fine detail). “Channels” here are the internal feature maps the network works with; more channels means more capacity.

Input and output. The input is a noisy image ($3 \times 32 \times 32$) together with the noise level t ; the output is the network’s estimate of the noise ε , with the same shape as the image. Predicting the *noise* (rather than the clean image) is the standard formulation of Ho et al. 2020, and it is called ε -*prediction*. it lets us predict the next image on the chain "noisy -> clear" if we know the noise difference to the next state.

Skip connections. Shortcuts link the matching levels of the down and up paths, so the fine details captured before compression are not lost when the image is rebuilt.

Channel multipliers. The width grows as we go deeper, controlled by the multipliers `channel_mults` (e.g. $64 \rightarrow 128 \rightarrow 128$).

Residual blocks (ResBlocks). The basic building block of each level; a residual (skip-inside-the-block) connection makes deep networks easier to train.

GroupNorm and SiLU. We normalise activations with Group Normalization and use the SiLU/Swish activation function, both standard choices for diffusion U-Nets.

Down/Up-sampling. Operations that reduce and increase the spatial resolution between levels of the U.

Time embedding. The network must behave differently depending on the noise level t to correctly predict the next step. We turn the single number t (from 0 to 999) into a vector of `time_dim= 256` numbers using sine and cosine functions at several frequencies (a *sinusoidal timestep embedding*), and we feed this vector into the layers. This tells the network “how much noise to expect”, so a single network can handle all 1000 levels.

Self-attention. Convolutions only look at nearby pixels, but good denoising also needs *global* coordination (the whole image should agree on what it is becoming). *Self-attention* lets every position in the image look at every other position and gather information from anywhere. Technically, for each position the network builds three vectors: a *Query* (“what am I looking for”), a *Key* (“what do I offer”) and a *Value* (“the information I carry”). Then it compares each Query with all Keys to decide how relevant every other position is, and then takes a weighted average of the Values. We use *multi-head* attention (several such comparisons in parallel, each able to focus on a different kind of relation) and we insert it only at the lower resolutions, because comparing every position with every other costs grow quadratically with the number of positions, so it is cheap only after the image has been compressed. Importantly, the Query/Key/Value/Output projections of these attention blocks are exactly where we later insert the LoRA adapters used by our novelty.

4.4 The training objective

For training, we take a real image, pick a random noise level t , add the corresponding amount of noise using Equation 1, and ask the network to guess the noise we added. The loss is the mean squared error (MSE) between the true noise and the predicted noise:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{x_0, t, \varepsilon} \|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2. \quad (2)$$

ε is the real noise we added, and $\varepsilon_\theta(x_t, t)$ is the network’s guess. The double bars with subscript 2 and power 2 mean “sum of the squared differences”. The symbol $\mathbb{E}$ (expectation) just means we average this error over many images, many noise levels, and many random noises. Minimising it teaches the network to denoise at every level. We will see that this loss drops quickly and then flattens, because most noise levels are “easy”; a low loss therefore does not by itself guarantee good images, so we always judge quality with FID.

4.5 Engineering optimizations we added around the baseline

The items above are the standard DDPM. However, we saw that with the resources given to us by Kaggle and Colab, we couldn’t produce images with high quality, because inserting the necessary parameters (e.g. higher number of channels) required a lot more computational power than we had.

INSERIRE IMMAGINI DI CONFRONTO PER MOSTRARE CHE AUMENTARE I PARAMETRI PRODUCE IMMAGINI MIGLIORI E HA SENSO CIÒ CHE é STATO DETTO.

Therefore, we decided to add a few optimizations to make training more stable, faster, fully reproducible, and have better quality with less time spent on the resources we had. They produce a solid infrastructure for the baseline.

EMA (Exponential Moving Average). During training we keep a second, “smoothed” copy of the weights, updated as $W_{\text{ema}} \leftarrow \text{decay} \cdot W_{\text{ema}} + (1 - \text{decay}) \cdot W$ with *decay* close to 1. Because the optimizer makes the raw weights wobble around a good configuration, this time-average is more stable and produces noticeably better images at no extra cost. We generate from the EMA weights, and the EMA model is also the *teacher* for distillation. On short runs a warm-up (scale the decay to not give early weights a disproportionate relevance) on the decay avoids the average staying anchored to the random initialization.

Mixed precision (AMP). Some computations are done in reduced (16-bit) precision with `torch.cuda.amp`, which makes training faster and lighter on memory with no visible loss of quality. A *GradScaler* rescales the gradients to avoid the numerical problems that reduced precision can cause.

Data augmentation. We apply a random horizontal flip to the training images. A mirrored CIFAR image is still a plausible image, so this enriches the data *without distorting* its distribution, unlike crops or rotations, which would generate unnatural images and worsen the FID.

Gradient clipping. We cap the size (norm) of the gradients before each update, so an occasional huge gradient cannot destabilise training.

Determinism and reproducibility. We fix the seed for all random sources (Python, NumPy, PyTorch, CUDA), enable deterministic algorithms (`use_deterministic_algorithms`, `cupy.deterministic`) and seed the data-loader workers, so re-running the code gives the same results.

Seeded shared noise bank. For evaluation we fix the initial random noise with a seeded generator, so *every* sampler denoises the same starting noise. This makes all comparisons paired: any difference in the output is due to the method, not to lucky noise.

Checkpointing and resume. We save the weights periodically (`ddpm_latest`) and at the end (`ddpm_final`, `ddpm_ema`), and each distilled student can resume from its own checkpoint.

4.6 Fast sampling with DDIM

Generating with the full DDPM means applying the network 1000 times. The standard to fasten things is DDIM. DDIM speeds this up by *skipping* levels in a deterministic way. One DDIM step from level t to a later level t' is:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}, \quad x_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}} \varepsilon_\theta(x_t, t). \quad (3)$$

First we use the predicted noise to *estimate the clean image* $\hat{x}_0$ (this is just Equation 1 solved for x_0). Then we re-noise that estimate down to the next level t' . The setting $\eta = 0$ makes the process deterministic (no extra random noise is added along the way). With fewer, larger jumps this is much faster, but each large jump introduces a small approximation error, and these errors add up, which is why very few steps hurt quality.

4.7 LoRA few-step distillation

What LoRA is. We said that DDPMs take a lot of time to make all 1000 steps for the generation of one image. DDIMs are one way to speed things up, but can hurt quality with few steps. So we decided to implement LoRA to distill a new model more capable.

Inside the attention layers, the network multiplies its input by large weight matrices W . Re-training all of them would be expensive. LoRA freezes W and adds next to it two *small* matrices A and B , whose product $B \cdot A$ represents the correction ΔW we want. Only A and B are trained. The number of intermediate channels in A and B is the *rank* (e.g. 4): a small rank means very few trainable numbers. During the forward pass the layer computes $y = Wx + (BA)x$: the frozen original plus a learned correction. We start with $B = 0$, so at the beginning the student behaves exactly like the teacher. At the end, A and B can be merged into W ($W' = W + BA$), so the model has the same shape as the original and costs nothing extra at inference.

The distillation objective. The trained baseline (EMA weights) is the frozen teacher ε_ϕ ; the student ε_θ is the same network with LoRA adapters. For a target step budget K we build a short schedule and pick a random interval ($t \rightarrow t'$). We form the starting state x_t by forward-diffusing a real image (with Equation 1). The *teacher* refines x_t from t to t' using m small DDIM sub-steps, giving a good target $x_{t'}^{\text{teacher}}$; the *student* must reach the same point in a *single* DDIM step, giving $x_{t'}^{\text{student}}$. The loss is:

$$\mathcal{L}_{\text{distill}} = \|x_{t'}^{\text{student}} - x_{t'}^{\text{teacher}}\|_2^2 + \lambda \|\varepsilon_\theta(x_t, t) - \varepsilon_\phi(x_t, t)\|_2^2. \quad (4)$$

The first term forces the student’s one-step jump to land where the teacher’s careful multi-step path lands: this is what teaches the student to compensate the error of large jumps. The second term (weighted by λ) is an auxiliary “noise-consistency” term that keeps the student’s noise prediction close to the teacher’s, which stabilises training. We train one student per target $K \in \{8, 4, 2, 1\}$, updating only the LoRA adapters.

Why this is efficient. LoRA does not make each denoising step faster. The speed-up comes from using fewer steps. LoRA’s role is to make the adaptation cheap to train (only $\sim 0.1\%$ of the parameters, so it fits on a single GPU) and free at inference (the adapters merge into the weights). So we get few-step sampling without the quality collapse of plain DDIM, and without the cost of full-model distillation.

5 Results

5.1 Baseline training

Figure 1 shows the training loss decreasing from about 0.089 to 0.032 and then flattening, as expected. Figure 2 shows uncensored samples from the baseline: the global structure (animals, vehicles, backgrounds) is visible, though fine detail is limited — consistent with a compact model trained under a single-GPU budget.

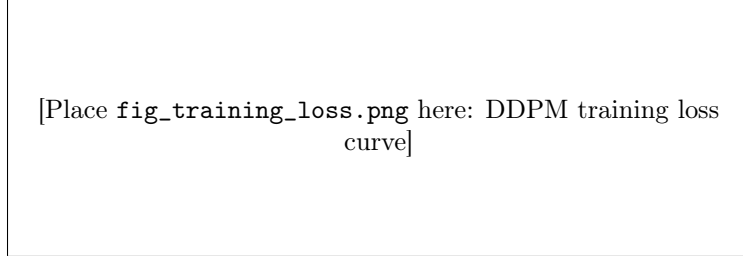


Figure 1: Baseline DDPM training loss over 80 epochs.

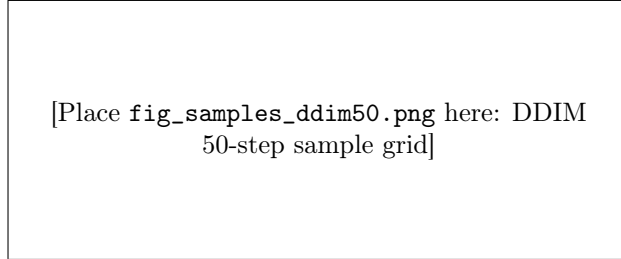


Figure 2: Uncensored baseline samples (DDIM, 50 steps).

5.2 Quality and speed vs. number of steps

Table 1 reports the baseline across step budgets. Fewer steps means faster but lower-quality sampling; crucially, **recall (diversity) collapses** long before precision does (Figure 4).

Table 1: Baseline: DDIM at different step counts (FID on 10 000 images). The DDPM row uses 1000 images and is reported only for speed-up factors.

Sampler	Steps	FID ↓	IS ↑	Precision	Recall	s/img	Speed-up [†]
DDPM (ref)	1000	67.3*	6.26	0.847	0.381	0.674	1×
DDIM	100	29.33	7.23	0.751	0.326	0.052	12.8×
DDIM	50	29.79	7.32	0.777	0.306	0.026	25.2×
DDIM	20	32.43	7.32	0.834	0.243	0.011	60.5×
DDIM	10	40.89	7.02	0.893	0.141	0.006	112.9×
DDIM	8	47.12	6.83	0.903	0.102	0.005	136.5×
DDIM	4	103.28	5.29	0.853	0.015	0.003	242.4×
DDIM	2	188.84	3.73	0.401	0.001	0.002	~360×
DDIM	1	325.11	1.56	0.005	0.000	0.001	~510×

[†] vs. DDPM-1000. * On 1000 images (noisier).

[Place `fig_fid_vs_steps.png` here: FID vs number of steps, DDIM and distilled points]

Figure 3: FID vs. number of denoising steps. DDIM (blue curve) degrades sharply below ~ 10 steps; the distilled LoRA students sit below the DDIM curve at the same step count.

[Place `fig_fidelity_diversity.png` here: precision vs recall as steps shrink]

Figure 4: Fidelity vs. diversity. As steps shrink, precision stays high while recall collapses: few-step sampling trades diversity for apparent fidelity.

Reading of Figure 4. As steps drop from 100 to 8, precision actually *rises* ($0.75 \rightarrow 0.90$) while recall *collapses* ($0.33 \rightarrow 0.10$, and to ~ 0 at 4 steps and below). Few-step sampling does not mainly make individual samples worse — it shrinks the portion of the data distribution the model covers. FID alone would hide this; Precision/Recall makes it explicit.

5.3 Novelty: matched-step comparison

Table 2 compares plain DDIM- K against the distilled LoRA- K at the same step budget, same initial noise, and same base weights — so the only difference is the LoRA adapter.

Table 2: Matched step budget K : plain DDIM vs. distilled LoRA.

K	DDIM FID	Distilled FID	Δ FID	DDIM Recall	Distilled Recall	Time ratio
8	47.12	38.24	+8.9	0.102	0.159	$1.01\times$
4	103.28	68.34	+34.9	0.015	0.051	$1.01\times$
2	188.84	159.12	+29.7	0.001	0.002	$1.01\times$
1	325.11	326.57	-1.5	0.000	0.000	$1.00\times$

The distilled student improves FID at $K = 8, 4, 2$, with the largest gain at $K = 4$ ($+34.9$), and partially **restores diversity** (recall $+0.057$ at $K = 8$). At $K = 1$ both models fail. Inference cost is unchanged (time ratio ≈ 1): the distilled $K = 8$ model is $\sim 10.6\times$ faster than DDIM-100 and the $K = 4$ model $\sim 18.9\times$ faster, at a fraction of the quality loss of plain DDIM at the same step count.

5.4 Ablation 1 (causal): distillation objective vs. naive eps-loss

To prove that the gain comes from the distillation *objective* and not simply from the extra LoRA parameters, we train an identical LoRA (same rank, same trainable parameters) with the standard noise-prediction loss (Eq. (2)) at low steps.

Table 3: Causal control: naive eps-loss LoRA vs. distillation, matched K and rank.

K	Naive-eps LoRA Δ FID	Distillation Δ FID
8	+0.96	+ 8.9
4	−3.51 (worse)	+ 34.9

The naive control gives ≈ 0 improvement at $K = 8$ and is *worse* than plain DDIM at $K = 4$, while distillation gives large gains. This isolates the trajectory-matching objective as the direct cause of the improvement.

5.5 Ablation 2: LoRA rank ($K = 8$)

FID: $r=2$: 38.32, $r=4$: 38.48, $r=8$: 37.67. Rank barely matters — even rank 2 captures almost all of the benefit, reinforcing the parameter-efficient claim (only $\sim 0.1\%$ of the weights are trained).

5.6 Ablation 3: distillation state source ($K = 8$)

Forward-diffused states only: 38.48; mixing teacher-trajectory states ($p = 0.5$): 38.24 — essentially tied. Conditioning on generated trajectory states does *not* help; forward-diffusion states are sufficient. We report this as a negative result and do not rely on trajectory-state mixing in the final method.

6 Limitations

- **Weak absolute baseline.** Under the single-GPU budget the baseline reaches FID ≈ 29 ; samples show global structure but limited detail. This is a deliberate hardware-driven choice; because all comparisons are relative to the same teacher, the conclusions about the novelty hold regardless of absolute quality.
- **Breakdown at $K = 1$.** The method does not recover single-step sampling on this baseline.
- **FID sample-size sensitivity.** We evaluate all headline comparisons at a fixed 10 000 images. The $K = 4$ result (+34.9) is well beyond this noise; the $K = 8$ result (+8.9) is smaller and would benefit from multiple seeds.
- **Linear β -schedule.** A cosine schedule could improve baseline quality and is left as future work.

7 Conclusion

We presented a parameter-efficient few-step distillation of a DDPM using LoRA adapters and a trajectory-matching objective. On CIFAR-10 the method improves few-step generation quality at a matched step budget (up to +34.9 FID at 4 steps), partially restores the diversity lost by aggressive step reduction, and adds no inference overhead, yielding ~ 10 – $19\times$ speed-ups over the DDIM-100 baseline (over $100\times$ vs. full DDPM). A naive-eps control confirms that the distillation objective — not the extra parameters — is responsible for the gain. Future work includes a cosine schedule, min-SNR-weighted distillation, and rank scaling for the extreme $K = 2/1$ regime.

References

- [1] J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS 2020.

- [2] J. Song, C. Meng, S. Ermon. *Denoising Diffusion Implicit Models*. ICLR 2021.
- [3] T. Salimans, J. Ho. *Progressive Distillation for Fast Sampling of Diffusion Models*. ICLR 2022.
- [4] E. Hu, Y. Shen, P. Wallis, et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021.
- [5] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, T. Aila. *Improved Precision and Recall Metric for Assessing Generative Models*. NeurIPS 2019.